

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

Звіт
про виконання лабораторної роботи №N3
« Створення власного класу. Методи розв'язку рівнянь»

Виконав
студент групи Феп-12
Носов А.О.

Львів-2026

Мета

Навчитися створювати власні класи у мові C++, реалізовувати методи класу та використовувати принципи інкапсуляції. Освоїти чисельні методи розв'язання нелінійних рівнянь, а саме метод дихотомії та метод Ньютона.

Хід роботи

Спочатку я створив заголовковий файл `Dyhotomia_class.h`, в якому оголосив клас для методу дихотомії. Клас містить три приватних поля: `a`, `b` та `eps`, що відповідають межах проміжку та точності обчислень. Для роботи з цими полями написав публічні методи `setVolumes` та `setTolerance`, а також метод `count`, який виконує сам розрахунок і записує результат у змінну за посиланням.

Далі створив файл реалізації `Dyhotomia_class.cpp`, де описав логіку методу дихотомії. Суть методу полягає в тому, що ми беремо проміжок, на кінцях якого функція має різні знаки, знаходимо середину і визначаємо, в якій половині знаходиться корінь. Цей процес повторюється поки довжина проміжку не стане менше заданої точності.

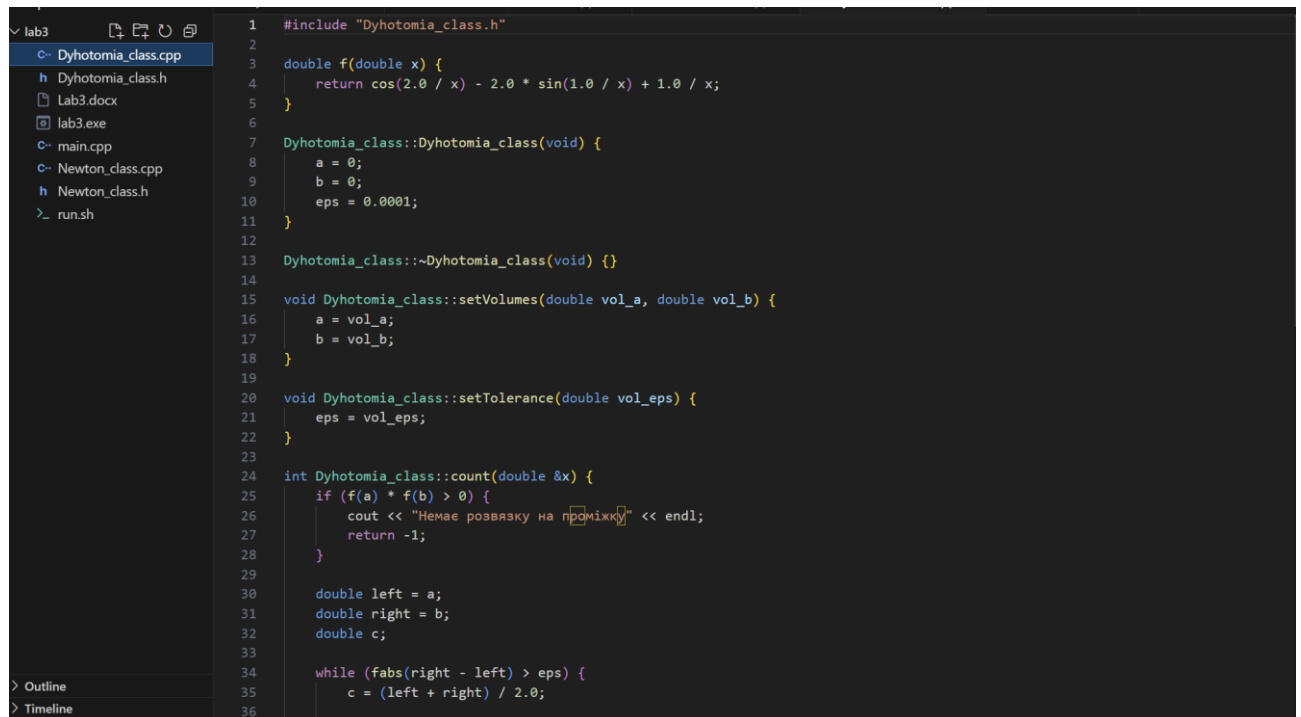
За аналогією створив клас `Newton_class` для методу Ньютона. Відмінність цього методу в тому, що він використовує дотичну до графіка функції для знаходження наступного наближення. Похідну обчислював чисельно за формулою скінченних різниць.

В основній програмі `main.cpp` підключив обидва класи, створив їх екземпляри через оператор `new`, задав параметри і запустив обчислення. Після отримання результатів об'єкти видалив через `delete`.

Для функції варіанту 5 $f(x) = \cos(2/x) - 2\sin(1/x) + 1/x$ на проміжку від 1 до 2 з точністю 0.0001 отримав наступні результати. Метод дихотомії знайшов корінь $x = 1.87564$ за 14 ітерацій. Метод Ньютона знайшов корінь $x = 1.87562$ за 4 ітерації.

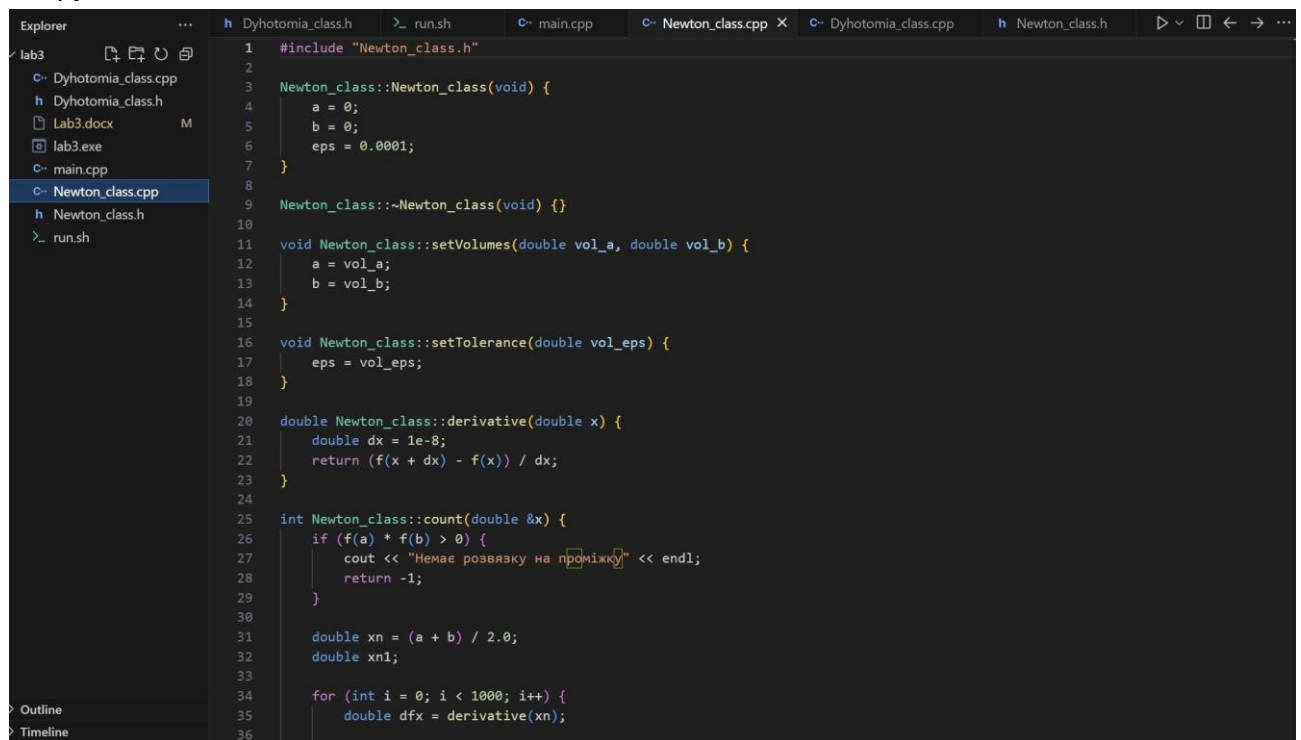
Фото з роботи

1. Перший клас



```
1 #include "Dyhotomia_class.h"
2
3 double f(double x) {
4     return cos(2.0 / x) - 2.0 * sin(1.0 / x) + 1.0 / x;
5 }
6
7 Dyhotomia_class::Dyhotomia_class(void) {
8     a = 0;
9     b = 0;
10    eps = 0.0001;
11 }
12
13 Dyhotomia_class::~Dyhotomia_class(void) {}
14
15 void Dyhotomia_class::setVolumes(double vol_a, double vol_b) {
16     a = vol_a;
17     b = vol_b;
18 }
19
20 void Dyhotomia_class::setTolerance(double vol_eps) {
21     eps = vol_eps;
22 }
23
24 int Dyhotomia_class::count(double &x) {
25     if (f(a) * f(b) > 0) {
26         cout << "Немає розв'язку на проміжку" << endl;
27         return -1;
28     }
29
30     double left = a;
31     double right = b;
32     double c;
33
34     while (fabs(right - left) > eps) {
35         c = (left + right) / 2.0;
```

2. другий клас



```
1 #include "Newton_class.h"
2
3 Newton_class::Newton_class(void) {
4     a = 0;
5     b = 0;
6     eps = 0.0001;
7 }
8
9 Newton_class::~Newton_class(void) {}
10
11 void Newton_class::setVolumes(double vol_a, double vol_b) {
12     a = vol_a;
13     b = vol_b;
14 }
15
16 void Newton_class::setTolerance(double vol_eps) {
17     eps = vol_eps;
18 }
19
20 double Newton_class::derivative(double x) {
21     double dx = 1e-8;
22     return (f(x + dx) - f(x)) / dx;
23 }
24
25 int Newton_class::count(double &x) {
26     if (f(a) * f(b) > 0) {
27         cout << "Немає розв'язку на проміжку" << endl;
28         return -1;
29     }
30
31     double xn = (a + b) / 2.0;
32     double xn1;
33
34     for (int i = 0; i < 1000; i++) {
35         double dfx = derivative(xn);
```

3. мейн

```
1  #include "Dyhotomia_class.h"
2  #include "Newton_class.h"
3
4  int main() {
5      system("chcp 65001>nul 2>nul");
6
7      double a, b, eps;
8
9      cout << "Введіть a: ";
10     cin >> a;
11     cout << "Введіть b: ";
12     cin >> b;
13     cout << "Введіть eps: ";
14     cin >> eps;
15
16     double x1, x2;
17
18     Dyhotomia_class *dyh = new Dyhotomia_class();
19     dyh->setVolumes(a, b);
20     dyh->setTolerance(eps);
21
22     if (dyh->count(x1) == 0) {
23         cout << "Дихотомія: x = " << x1 << endl;
24         cout << "f(x) = " << f(x1) << endl;
25     }
26
27     delete dyh;
28
29     Newton_class *newt = new Newton_class();
30     newt->setVolumes(a, b);
31     newt->setTolerance(eps);
32
33     if (newt->count(x2) == 0) {
34         cout << "Ньютон: x = " << x2 << endl;
35         cout << "f(x) = " << f(x2) << endl;
36     }
37
38     delete newt;
39
40     return 0;
41 }
42
```

4. вивід

```
artno@vivobook MINGW64 ~/OneDrive - lnu.edu.ua/OOP/lab3 (main)
$ bash run.sh
Введіть a: 1
Введіть b: 2
Введіть eps: 0.001
Дихотомія: x = 1.87549
f(x) = -9.07776e-05
Ньютон: x = 1.87562
f(x) = -1.78274e-11
```

Висновок

Під час виконання лабораторної роботи я навчився створювати власні класи в C++ та застосовувати принципи інкапсуляції, коли дані класу захищені від прямого доступу і змінюються тільки через спеціальні методи. Реалізував два методи розв'язання нелінійних рівнянь. Метод дихотомії простіший у реалізації, але потребує більше ітерацій. Метод Ньютона збігається значно швидше, проте вимагає обчислення похідної. Обидва методи дали практично однаковий результат, що підтверджує їх коректність.